Capítulo 1

Engenharia de IA é engenharia

1. A pergunta de engenharia

A maioria dos projetos de IA começa com uma tecnologia procurando um lugar onde ser usada. Uma equipe ganha acesso a um modelo capaz, alguém pergunta onde ele poderia ser usado e, em uma semana, existe uma lista de funcionalidades que poderiam "adicionar IA". Algumas dessas funcionalidades dão certo. Muitas acabam sendo maneiras caras de fazer algo que um componente mais simples já fazia, ou teria feito, a um custo menor e com menos surpresas.

A pergunta por trás dessa lista é:

> "Onde podemos usar um LLM?"

Ela parece prática e é o ponto de partida errado, pelo mesmo motivo que "onde podemos usar uma fila de mensagens?" é o ponto de partida errado. Começa pela solução e sai procurando problemas que se encaixem nela.

Este livro parte de outra pergunta:

Que problema estamos tentando resolver, e qual é a tecnologia mais simples que o resolve bem o suficiente?

Essa pergunta não é um argumento contra a IA. Às vezes a resposta é um modelo de linguagem, e às vezes nada mais simples serve. O ponto é que a resposta vem *depois* que o problema foi enunciado, as restrições foram nomeadas e as alternativas foram comparadas. Partir do problema não exclui a IA. Coloca a IA na mesma posição de qualquer outra tecnologia: uma escolha que precisa conquistar seu lugar, não uma premissa de partida.

Modelos de linguagem tornam essa disciplina mais difícil de manter, por motivos que vão aparecer muitas vezes neste livro:

- **São fáceis de chamar.** Adicionar uma chamada ao modelo muitas vezes leva poucas linhas de código. A facilidade da chamada esconde o custo de tudo o que existe ao redor dela: latência, gestão do modelo, avaliação, tratamento de falhas.
- **A saída deles é persuasiva.** Uma resposta fluente parece correta. Uma demo que funciona em três exemplos parece uma funcionalidade.
- **São probabilísticos.** O mesmo componente pode fazer a coisa certa com uma entrada e a errada com uma entrada parecida, e a causa muitas vezes não é visível no código.
- **São parcialmente observáveis.** Você vê o que entrou e o que saiu. Em geral, não consegue ver por que uma saída foi produzida em vez de outra.

Nenhuma dessas propriedades faz da engenharia de IA uma disciplina diferente. Elas tornam a disciplina comum da engenharia — enunciar o problema, saber o que precisa ser verdade, reunir evidências, comparar alternativas, decidir e saber quando reavaliar — mais necessária, não menos.

O estudo de caso deste capítulo é pequeno e real. Uma regra simples de palavras-chave decidia se cada requisição a um dos pontos de entrada do sistema do laboratório deveria receber uma resposta ou um diagrama. Por um tempo, esse foi o design certo. Então um usuário anexou uma imagem e perguntou o que havia nela, e a regra mandou a requisição pelo caminho errado. A correção substituiu a regra por uma chamada ao modelo, o que acrescentou uma chamada ao modelo a cada requisição.

O caso não está neste capítulo porque o modelo venceu. Está aqui porque mostra as perguntas de que este livro trata: quando uma regra determinística deixa de ser suficiente para um problema, quanto custa a alternativa e o que você sabe e não sabe depois de fazer a mudança.

Ao fim do capítulo, você deve ser capaz de:

- enunciar um problema antes de escolher uma tecnologia, e dizer o que “bem o suficiente” significa para ele;
- dizer se uma parte do seu sistema precisa *verificar correspondência* ou *interpretar*;

- listar os custos que um componente probabilístico acrescenta além da própria chamada;
- descrever o ciclo de engenharia que o restante do livro aplica, e dizer por quais de suas etapas uma determinada decisão de fato passou.



2. Conceitos centrais

Engenharia a partir do problema

O enunciado do problema é o artefato mais subestimado de um projeto de IA. Ele responde a quatro perguntas antes que qualquer tecnologia seja mencionada:

- **Qual é a tarefa?** O que o sistema deve fazer, para quem, em quais situações?
- **O que significa "bem o suficiente"?** Que qualidade, latência e custo tornariam o resultado aceitável? Que falha o tornaria inaceitável?
- **O que precisa ser sempre verdade?** Quais restrições não são negociáveis: quem pode ver quais dados, quais ações são permitidas, o que nunca deve ser enviado para fora do sistema?
- **Como vamos saber?** Que observação ou medição nos diria que o problema foi resolvido?

Uma equipe que não consegue responder a essas perguntas não está pronta para escolher uma tecnologia. Ainda pode prototipar, e protótipos são úteis para aprender qual é o problema. Mas um protótipo que "funciona" é evidência sobre o protótipo, não sobre se o problema foi resolvido.

> *Ilustrativo.* “Adicionar um assistente de IA à página de suporte” é uma solução. “Reduzir o número de chamados de suporte que pedem informações que já estão na documentação, sem dar aos usuários respostas erradas sobre cobrança” é um problema. Só o segundo diz o que medir, o que proteger e quais alternativas comparar. Uma caixa de busca melhor, uma página de documentação reescrita e um modelo de linguagem são todos candidatos para ele.

Engenharia a partir do problema não significa que o enunciado do problema seja fixo. Problemas mudam, e o estudo de caso deste capítulo trata exatamente disso. Quando o problema muda, reexamine a tecnologia diante do novo problema em vez de remendar a solução antiga.

A tecnologia mais simples que resolve o problema bem o suficiente

O princípio ao qual este livro sempre retorna é:

Use a tecnologia mais simples que resolve o problema bem o suficiente.

Três palavras concentram o peso da ideia.

Mais simples não significa “sem IA”. Significa a opção com o menor custo, a menor complexidade e a menor incerteza adicionais *para este problema*. Às vezes é uma tabela de consulta ou um índice de busca. Às vezes é uma chamada ao modelo, porque as alternativas precisariam de tantas regras e exceções que seriam mais complexas do que o modelo que evitam.

Resolve significa que a opção de fato executa a tarefa, não que ela lida com os exemplos que alguém testou. Essa afirmação precisa de evidência, e reunir essa evidência é o assunto do Capítulo 2.

Bem o suficiente significa que existe um limiar, e que ele foi definido antes de as opções serem comparadas. Sem um limiar, qualquer opção pode ser apresentada como adequada ou inadequada depois do fato.

O princípio vale nos dois sentidos. Ele argumenta **contra** adicionar um modelo onde uma regra, uma query ou um formulário já resolve o problema bem o suficiente, e **a favor** de um modelo onde o problema

exige algo que regras não conseguem expressar. Ele não decide nada sozinho. Ele diz qual comparação fazer.

Componentes determinísticos e probabilísticos

A distinção mais útil deste livro é entre dois tipos de componentes.

Um **componente determinístico** produz a mesma saída para a mesma entrada, por regras que alguém escreveu, desde que seu estado, sua configuração e outras condições relevantes permaneçam os mesmos. Uma verificação por palavra-chave, uma query de banco de dados, uma função de validação e uma verificação de permissão são determinísticas. Você consegue ler as regras, enumerar seus casos e, quando o domínio de entrada é finito e tratável, testá-las exaustivamente. Determinístico não significa correto. Quando um componente determinístico está errado, ele erra da mesma forma toda vez que a mesma entrada aparece, e a causa está nas regras ou em quão bem elas se ajustam ao problema.

Um **componente probabilístico** produz saídas a partir de um modelo aprendido, e não de regras que alguém escreveu. Um modelo de linguagem é o principal exemplo neste livro: a cada passo, ele produz uma distribuição de probabilidade sobre as saídas possíveis, e o runtime escolhe a partir dela (Capítulo 3). Probabilístico não significa aleatório; suas saídas são moldadas pela entrada e muitas vezes são altamente consistentes. Configurações como a temperatura podem tornar sua saída repetível em uma dada configuração de execução (Capítulo 3), mas repetível não é o mesmo que especificado: você continua sem conseguir dizer, de antemão, o que ele fará com uma entrada que ninguém testou.

Nenhum dos dois tipos é melhor em geral. Componentes determinísticos são previsíveis, baratos de executar e simples de testar, mas só fazem o que suas regras dizem. Componentes probabilísticos podem lidar com entradas que ninguém previu — paráfrases, ambiguidade, contexto, intenção —, mas também podem falhar nelas, e se falham ou não é algo que se descobre medindo, não lendo o código.

A diferença que importa para a engenharia é **o que você precisa fazer para confiar em cada um**:

	Componente determinístico	Componente probabilístico
Especificação	As regras são a especificação	O comportamento é especificado apenas por exemplos e instruções
Testes	Os casos podem ser enumerados a partir das regras	O comportamento precisa ser amostrado; os testes mostram o que aconteceu nos casos testados
Falha	Repete-se na mesma entrada; a causa está nas regras	Pode depender da formulação ou do contexto; a causa pode não ser visível
Evidência de qualidade	Ler as regras e testar seus casos	Medir em um conjunto representativo de casos (Capítulos 10 e 11)
Incerteza residual	Principalmente sobre se as regras correspondem ao problema	Sobre a taxa e o tipo de erros, que precisam ser estimados

Quando você substitui um componente determinístico por um probabilístico, não muda apenas o que o sistema consegue fazer. Muda o que você precisa medir, testar e observar, e aceita um tipo de incerteza que antes não existia. Muitas vezes essa é a troca certa. Ela deve ser sempre visível.

Esta é uma distinção de trabalho para a engenharia, não uma taxonomia em que todo componente se encaixa perfeitamente. Sistemas reais misturam os dois tipos: uma regra com fallback para um modelo, um modelo restrito a um conjunto fixo de saídas, uma verificação determinística sobre a saída de um modelo. A distinção é útil porque aponta o que precisa ser medido.

Correspondência versus interpretação

Uma forma prática de dizer de que tipo de componente uma parte do seu sistema precisa é perguntar se a tarefa é de **correspondência** ou de **interpretação**.

Correspondência pergunta se uma entrada contém algo que pode ser escrito, é igual a esse algo ou o satisfaz: uma palavra, um código, um formato, um valor dentro de uma faixa. Regras são boas em correspondência.

Interpretação pergunta o que uma entrada *significa*: o que o usuário quer, se duas formulações dizem a mesma coisa, se um trecho sustenta uma afirmação. Esse é um problema diferente, mesmo quando parece correspondência. Regras podem aproximar a interpretação, muitas vezes surpreendentemente bem, enquanto as entradas permanecerem próximas do que o autor da regra tinha em mente.

O problema começa quando uma regra é usada para aproximar a interpretação e as entradas mudam: novas funcionalidades, novos usuários ou novos tipos de conteúdo trazem entradas que o autor nunca imaginou. A regra continua correspondendo exatamente àquilo para o que foi escrita. Aquilo a que ela corresponde já não significa o que significava antes. Nada na regra mudou, e nada no código sinaliza a falha. O estudo de caso deste capítulo é um exemplo claro disso.

Complexidade é custo

Toda opção em uma comparação de design tem custos; regras precisam ser mantidas, e perguntar ao usuário acrescenta interface. Um componente probabilístico traz custos fáceis de ignorar, porque a chamada em si é fácil de escrever. Ele pode acrescentar:

- **latência:** mais uma chamada a esperar, muitas vezes em toda requisição;
- **custo:** computação ou cobrança por chamada;
- **dependências operacionais:** um modelo para hospedar e versionar, ou um provedor do qual depender;
- **modos de falha:** timeouts, modelos indisponíveis, saídas malformadas, saídas erradas que parecem certas;
- **requisitos de observabilidade:** registrar entradas, saídas e decisões para que as falhas possam ser diagnosticadas;

- **requisitos de avaliação:** uma forma de saber se o componente é bom o suficiente, e se ele piorou;
- **incerteza:** uma taxa de erro que precisa ser estimada em vez de lida no código.

Nenhum desses custos é motivo para evitar a IA. Eles pertencem à comparação, ao lado dos custos das alternativas, antes de a decisão ser tomada, e não devem ser descobertos depois que ela vai para produção. O Capítulo 12 trata latência e custo como insumos de projeto medidos, e o Capítulo 14 mostra como combiná-los com qualidade e restrições em uma única decisão.

O método de engenharia

Este livro aplica um único ciclo a toda decisão:

Problema → Hipótese → Baseline → Experimento → Medir → Comparar → Decidir → Observar → Iterar

Em resumo: enuncie o **problema** e o que "bem o suficiente" significa; transforme uma crença sobre o que vai resolvê-lo em uma **hipótese** que uma observação específica poderia refutar; estabeleça um **baseline** de como o sistema se comporta hoje; execute um **experimento** que muda uma única coisa; **meça** o que aconteceu e em quais condições; **compare** o resultado com o baseline e com o limiar, incluindo os custos; **decida**, e registre por quê; **observe** a decisão em uso; e **itere** quando ela deixar de se ajustar.

O ciclo importa por três motivos.

Ele coloca a decisão depois da evidência. Implementar primeiro e medir depois transforma qualquer resultado em justificativa para algo que já foi construído.

Ele torna as lacunas visíveis. Decisões reais muitas vezes pulam etapas: nenhum baseline, um experimento com um único caso, ninguém observando. Etapas nomeadas permitem dizer quais foram puladas, e esse é o primeiro passo para saber o quanto se pode confiar em uma decisão.

Ele não termina. Uma decisão bem tomada para um problema pode se tornar errada quando o problema muda, sem nenhuma mudança no código.

Este capítulo mostra por que o ciclo importa, e seu estudo de caso percorre o ciclo inteiro em uma decisão pequena, incluindo as etapas pelas quais ela não passou. Como executar o meio do ciclo é o assunto do Capítulo 2; o Capítulo 14 retorna a “decidir”.

O sistema do laboratório

A evidência deste livro vem de um único sistema, chamado de **sistema do laboratório** ao longo de todo o texto. Ele é apresentado aqui uma única vez. Nos capítulos seguintes, aparece em registros de Lab Notebook, e não como assunto dos capítulos.

O sistema do laboratório é uma plataforma multi-tenant de perguntas e respostas sobre documentos. Os usuários fazem perguntas sobre os documentos da sua organização. O sistema recupera trechos relevantes dos documentos daquele tenant, um modelo de linguagem gera uma resposta com citações, e uma segunda chamada ao modelo verifica se a resposta é sustentada pelos trechos. Ele executa modelos locais em um host somente com CPU, com um fallback opcional para um modelo hospedado nos casos que o caminho local não consegue responder. Seus prompts estão em português no sistema original, e esta edição os traduz.

O laboratório existe para fornecer **evidência de engenharia real e delimitada**: decisões que foram de fato tomadas, medições que foram de fato feitas, falhas que de fato aconteceram, documentadas em registros de decisão e registros de investigação que os registros de Lab Notebook citam como fontes. Não é uma arquitetura de referência nem uma recomendação. As tecnologias que ele por acaso usa aparecem apenas como contexto de seus experimentos, nunca como requisitos.

Todo resultado de laboratório neste livro é delimitado por suas condições: um sistema, modelos e versões específicos, um tipo de hardware, conjuntos de tarefas e tamanhos de amostra específicos, muitas vezes pequenos. Os rótulos de evidência mantêm esses limites visíveis. Este capítulo usa três dos cinco rótulos definidos no Capítulo 2: *Observado no laboratório*, *Não medido neste projeto* e *Ilustrativo*.

3. Opções e trade-offs

Quando parte de um sistema precisa decidir algo sobre sua entrada — qual caminho uma requisição segue, a que categoria uma mensagem pertence, se um documento é relevante —, há várias formas amplas de construir essa decisão. Nenhuma está sempre certa.

Opção	O que ela oferece	O que ela custa	Principal risco
A. Regras fixas (palavras-chave, padrões, tabelas de consulta)	Previsíveis, rápidas, baratas, testáveis pela leitura das regras	Regras a manter; só lidam com o que o autor previu	Erradas da mesma forma, toda vez, em entradas que as regras não previram
B. Perguntar ao usuário (pontos de entrada separados, escolhas explícitas, formulários)	Nenhuma inferência necessária; o usuário declara a intenção	Mais interface; os usuários precisam saber qual opção querem	Os usuários escolhem errado ou não querem escolher
C. Regras com remendos (adicionar exceções à medida que surgem falhas)	Mantém o componente determinístico; corrige o caso relatado	Cada remendo corrige uma entrada, não a classe de entradas	A regra cresce até que ninguém consiga prevê-la; a próxima falha não é evitada
D. Um classificador probabilístico (uma chamada ao modelo ou um pequeno classificador treinado)	Pode interpretar significado, paráfrase e contexto	Latência, custo, dependências, modos de falha, avaliação	Uma taxa de erro que precisa ser medida e não pode ser lida no código

Opção	O que ela oferece	O que ela custa	Principal risco
E. Regras primeiro, modelo para o resto	O custo do modelo é pago só onde as regras não têm certeza	Dois mecanismos a manter e testar, e uma fronteira entre eles	A regra continua decidindo com confiança, e de forma errada, nas entradas que interpreta mal

Cada opção é válida para algum problema. **Regras fixas costumam ser a primeira resposta certa**: uma regra simples, rápida e correta nas entradas que o sistema realmente recebe é difícil de superar. **Remendar regras indefinidamente** parece prudente, mas cada remendo corrige uma entrada, e não a classe de falha, e a regra fica mais difícil de prever com o tempo. **Um classificador probabilístico muda a evidência exigida**: "a decisão está correta?" deixa de ser uma pergunta que se responde lendo código e passa a ser uma que se responde medindo.

A favor e contra significam aqui o mesmo que em todo o livro. A tabela argumenta *a favor* de começar com regras onde elas bastam, o que não significa recomendá-las em toda parte. Ela argumenta *contra* remendar regras indefinidamente, o que não significa rejeitar regras. A escolha depende do problema, e o problema pode mudar.

4. Lab Notebook

Os registros abaixo usam o formato Lab Notebook definido no Capítulo 2. O prompt de roteamento do laboratório e seus rótulos estão em português no sistema original; esta edição os traduz.

Uma regra de palavras-chave que estava certa até o problema mudar

LAB NOTEBOOK 1.1 — Do roteamento por palavras-chave à classificação por modelo

Status da evidência Observado no laboratório

Fonte Registros de decisão do ponto de entrada único com roteamento por palavras-chave (ADR 0006, incluindo sua atualização posterior) e do roteamento baseado em modelo (ADR 0024); anexo de imagem em perguntas (ADR 0023)

Contexto O ponto de entrada único do sistema do laboratório aceita uma pergunta ou um pedido de diagrama, e precisa decidir qual caminho cada requisição segue. O primeiro design usava uma lista fixa de palavras-chave: depois da remoção de acentos, o texto da requisição era verificado em busca de palavras associadas a pedidos de recursos visuais (por exemplo “diagram”, “flow”, “architecture”, “chart” e “imagem”). Qualquer correspondência mandava a requisição para a geração de diagrama; caso contrário, ela ia para a geração de resposta. Mais tarde, uma mudança separada permitiu que os usuários anexassem uma imagem a uma pergunta.

Metadados registrados / ausentes Registros de decisão e histórico de código registrados (roteamento por palavras-chave introduzido em 2026-07-24; roteamento baseado em modelo em 2026-07-29). O modelo de classificação não é identificado no registro além de “the resolved chat model”. Versão do runtime e hardware não registrados na fonte.

Pergunta / Hipótese Não foi um experimento planejado. A pergunta levantada pelo incidente foi: a regra de palavras-chave ainda é suficiente para o roteamento, agora que os usuários podem anexar imagens e perguntar sobre elas?

Método A requisição de um usuário real falhou. A pergunta que falhou foi reproduzida com uma imagem de teste anexada, antes e depois da mudança.

Tamanho da amostra Um incidente real; uma reprodução da pergunta que falhou, antes e depois da mudança.

Resultado *Observado:* (1) O registro original descrevia o roteamento como uma simples verificação de substring, e não como uma chamada extra ao modelo, para que ele não acrescentasse latência nem um modo de falha adicional; o registro posterior o descreve como escolhido deliberadamente em vez de uma chamada ao modelo, para não acrescentar nenhuma latência. O registro original também previa sua fraqueza: uma pergunta que por acaso contivesse uma palavra correspondente seria roteada para a geração de diagrama mesmo quando se queria uma resposta em texto. Ele aceitava esse

> trade-off na primeira versão e nomeava a substituição a considerar caso o roteamento por falso positivo se tornasse um problema real: um prompt de classificação barato ou um pequeno classificador local. (2) Depois que o anexo de imagens entrou em produção, um usuário real anexou uma foto e perguntou “O que tem nessa imagem?”. A pergunta continha “imagem”, uma das palavras-chave. A requisição foi roteada para a geração de diagrama e devolveu um diagrama construído a partir de documentos recuperados sem relação com a pergunta, em vez de uma descrição da foto. (3) A lista de palavras-chave foi removida. O roteamento agora faz uma chamada de classificação com temperatura 0 que pede ao modelo que responda com exatamente uma de duas palavras, uma para diagrama e outra para resposta. O prompt afirma que palavras incidentais como “image”, “diagram” ou “architecture” não indicam, por si só, um pedido de diagrama, e que perguntas sobre o conteúdo de uma imagem anexada devem ser respondidas. (4) Na reprodução, a mesma pergunta com uma imagem de teste anexada produziu um diagrama antes da mudança e uma descrição da imagem depois dela. (5) O ponto de entrada único agora faz **uma chamada adicional ao modelo em toda requisição**. Os endpoints de propósito único originais, que nunca passaram por essa verificação de roteamento, não foram afetados.
>
> **Interpretação** A regra de palavras-chave não quebrou. Ela continuou correspondendo exatamente àquilo para o que foi escrita. O problema mudou: quando os usuários passaram a poder anexar imagens, a palavra “imagem” deixou de indicar um pedido de figura e passou a aparecer na pergunta mais natural sobre um anexo. A tarefa sempre tinha sido interpretar a intenção; por um tempo, a correspondência de palavras a aproximou bem o suficiente. Um novo tipo de entrada fez a aproximação falhar, e a condição de reavaliação do próprio registro foi atendida.
>
> **Decisão** Substituir o roteamento por palavras-chave por uma chamada de classificação ao modelo, aceitando uma chamada adicional ao modelo por requisição em troca de rotear pela intenção, e não por palavras incidentais. A chamada de roteamento passa pelo mesmo caminho de tratamento de falhas das demais chamadas ao modelo do sistema, de modo que uma indisponibilidade do modelo afeta o roteamento da mesma forma que afeta a geração. Um teste de regressão fixa a pergunta exata que falhou.
>
> **Limitações** Um incidente e uma reprodução; trata-se de uma decisão de engenharia registrada, não de uma medição. O classificador não é perfeito: o próprio registro de decisão observa que ele pode julgar mal uma pergunta incomumente ambígua. *Não medido neste projeto:* a acurácia do roteamento antes ou depois da mudança, em qualquer conjunto de perguntas; o impacto na latência e o impacto no custo da chamada adicional.

Duas coisas neste registro são fáceis de deixar passar.

O primeiro design não foi um erro. Ele escolheu a tecnologia mais simples que resolvia o problema naquele momento, e registrou tanto a falha prevista quanto o que fazer se essa falha se tornasse real.

O que mudou foi a entrada, não o código. O anexo de imagens era uma funcionalidade separada, com seu próprio registro de decisão, e nada nela tocava o roteamento. Mas ela mudou as entradas que o roteamento recebia. Funcionalidades interagem pelas entradas que criam, não só pelo código que compartilham.

O que a mudança deixou em aberto

A decisão trocou um tipo de incerteza por outro. A regra de palavras-chave tinha uma classe de falha conhecida e garantida: qualquer pergunta que contivesse uma de suas palavras ia para diagramas, toda vez. O classificador pode julgar mal algumas perguntas, mas quais, e com que frequência, não é visível no código. O registro de decisão chama isso de um risco qualitativamente diferente. A mudança tratou a falha observada e roteou corretamente o caso reproduzido; a decisão foi aceita apesar da lacuna de medição descrita abaixo.

> **LAB NOTEBOOK 1.2 — Acurácia do roteamento (variante Não medido)**
>
> **Status da evidência** Não medido neste projeto
>
> **Pergunta** Com que acurácia cada abordagem de roteamento envia as requisições para o caminho pretendido, e quanto a mudança custa em latência e em computação?
>
> **O que responderia à pergunta** Um conjunto rotulado de requisições reais cobrindo as duas intenções, incluindo perguntas sobre anexos, pedidos de diagrama formulados sem as palavras-chave e perguntas que mencionam palavras relacionadas a diagramas de forma incidental. As duas abordagens executadas sobre o mesmo conjunto, com a acurácia relatada por categoria e a latência da etapa de roteamento medida no mesmo host.
>
> **O que temos no lugar** O único incidente e a reprodução do Notebook 1.1, a previsão do registro original sobre a classe de falha da regra de palavras-chave e um teste de regressão para a pergunta que falhou. O Capítulo 5 (Notebook 5.3) registra a mesma lacuna.
>
> **Risco de decidir sem isso** A decisão se apoia na natureza da falha, um erro de roteamento garantido em uma pergunta comum, e não em uma melhoria medida. Ela não mostra que o classificador roteia melhor no geral, com que

> frequência ele erra o roteamento nem quanto custa a chamada adicional. Um pedido genuíno de diagrama enviado para o caminho de resposta não seria detectado por nada do que foi descrito aqui.

Muitas decisões reais são tomadas assim: uma única falha clara, uma correção que trata sua causa e uma medição que nunca acontece porque o problema visível desapareceu. O Capítulo 10 mostra o que essa medição envolveria.

O caso ao longo do ciclo

Etapa	O que aconteceu	Quão completa
Problema	Um ponto de entrada para dois tipos de requisição; rotear cada uma corretamente	Enunciado no primeiro registro de decisão
Hipótese	O primeiro design: as palavras-chave são específicas o suficiente para pedidos de recursos visuais. Depois: um classificador vai rotear pela intenção onde as palavras enganam	Enunciada, com a falha prevista do primeiro design
Baseline	O comportamento da regra de palavras-chave na pergunta que falhou	Um caso, não uma taxa
Experimento	A pergunta que falhou, reproduzida com uma imagem anexada, antes e depois	Um caso
Medir	Diagrama antes, descrição depois	Observado, não medido em um conjunto

Etapa	O que aconteceu	Quão completa
Comparar	A classe de falha conhecida contra uma taxa de erro não medida; uma chamada adicional por requisição	Qualitativo; custos não medidos
Decidir	Substituir a regra; aceitar a chamada adicional	Registrado, com o trade-off enunciado
Observar	A condição de reavaliação do primeiro design foi atendida pela requisição de um usuário real	O gatilho funcionou
Iterar	A decisão foi reavaliada, e um teste de regressão foi adicionado	Feito uma vez; acurácia não acompanhada

A tabela não é uma crítica à decisão. Algumas etapas são fortes, algumas se apoiam em um único caso e uma está ausente. Saber qual é qual é o que permite dizer até onde se pode confiar na decisão.

5. Guia de decisão

Antes de introduzir um componente de IA

1. Qual é o problema, enunciado sem nomear uma tecnologia?
2. Que limiar define “bem o suficiente”, e que latência e que custo você pode pagar?
3. O que precisa ser sempre verdade?
4. Qual é a opção mais simples que poderia superar o limiar?
5. A tarefa exige correspondência ou interpretação?
6. O que o componente de IA acrescentaria (Seção 2, “Complexidade é custo”)?
7. Que evidência vai dizer que ele funciona?

Sinais de que uma regra determinística não basta mais

- A regra verifica a correspondência de palavras para inferir significado, e as palavras começaram a aparecer por outros motivos.
- Uma nova funcionalidade mudou as entradas que a regra recebe.
- Cada falha é corrigida com a adição de uma exceção.
- A regra falha toda vez em uma entrada comum e natural.
- A condição de reavaliação que você registrou foi atendida.

Ao adicionar um componente probabilístico, registre

- o que ele oferece: a classe de falha que elimina ou a capacidade que acrescenta;
- o que ele custa, medido ou explicitamente marcado como não medido;
- o que se torna incerto, e quais erros passariam despercebidos;
- o que acontece quando ele está indisponível ou devolve algo inutilizável;
- que evidência você tem, e com qual rótulo;
- o que faria você reavaliar a decisão.

6. Limites

Este capítulo estabelece uma forma de pensar. Sua evidência é deliberadamente pequena.

Um caso observado, não um benchmark. O estudo de caso é uma única falha real e uma única reprodução. Ele não mostra que classificadores roteiam melhor do que regras de palavras-chave em geral, nem neste sistema como um todo. A única afirmação quantitativa que ele sustenta é arquitetural: uma chamada adicional ao modelo por requisição. Acurácia do roteamento, impacto na latência e impacto no custo: *não medido neste projeto* (Notebook 1.2).

Um sistema, um cenário de idioma. A lista de palavras-chave do laboratório misturava palavras em português e em inglês, e seu prompt de roteamento está em português. Não se sabe se a mesma regra teria falhado, ou se o mesmo classificador teria funcionado, em outro idioma ou em outro sistema.

O princípio não é uma fórmula. "A tecnologia mais simples que resolve o problema bem o suficiente" diz qual comparação fazer. Ele

não faz a comparação, não define o limiar nem mede as opções. Os Capítulos 2, 10 e 14 tratam dessas etapas.

7. Para ir além

Este capítulo argumentou por que um engenheiro de IA deve pensar assim. Ele não mostrou como produzir a evidência que falta ao seu próprio estudo de caso: um baseline com o qual valha a pena comparar, um experimento capaz de refutar uma correção, uma medição que mude uma decisão.

Capítulo 2 — Evidência antes de opinião: como conduzir um experimento de engenharia responde a essa questão. Ele transforma o meio do ciclo — hipótese, baseline, experimento, medir, comparar — em um método prático, apresenta os cinco rótulos de evidência e o formato Lab Notebook, e usa o sistema do laboratório para mostrar três correções propostas medidas antes que qualquer uma delas fosse implementada.

Os capítulos seguintes aplicam a mesma postura a cada parte de um sistema de IA:

- **Parte II — Modelos** (Capítulos 3 e 4): o que um modelo de linguagem oferece e o que não oferece, e como escolher um.
- **Parte III — Moldando comportamento e conhecimento** (Capítulos 5 a 8): prompting, embeddings e Retrieval, RAG como decisão de engenharia e Context Engineering. O Capítulo 5 retoma o prompt de roteamento do estudo de caso deste capítulo.
- **Parte IV — Sistemas que agem** (Capítulo 9): Tool Calling e primeiros agentes.
- **Parte V — Como saber se funciona** (Capítulos 10 e 11): como avaliar sistemas de IA e como decidir quando um benchmark merece confiança.
- **Parte VI — Trade-offs** (Capítulos 12 a 14): latência e custo como insumos de projeto, fundamentos de segurança e a tomada de decisão.

Livro 2 — Production AI Engineering trata da operação de sistemas de IA em produção. **Livro 3 — AI Model Lab** trata de modelos e da ciência da avaliação em profundidade.